Machine Learning-Based Cybersecurity Attack Detection Using Ensemble Learning Techniques

A. Varsha
UG, CSE-AI, FET
Jain (Deemed-to-be-University)
Bangalore-562112, India

A. Hema
UG, CSE-AI, FET
Jain (Deemed-to-be-University)
Bangalore-562112, India

V. Rajeshwari
UG, CSE-AI, FET
Jain (Deemed-to-be-University)
Bangalore-562112, India

C. Akshaya
UG, CSE-AI, FET
Jain (Deemed-to-be-University)
Bangalore-562112, India

***Abstract*—The rapid increase in cyber threats and network-based attacks has necessitated the development of efficient and intelligent intrusion detection systems (IDS). This paper presents a machine learning-based framework for cybersecurity attack detection using a large-scale network traffic dataset with diverse features. Data preprocessing techniques, including missing value imputation and feature scaling, are applied to enhance data quality and model performance.**

A comprehensive set of classification algorithms, namely Decision Tree, Random Forest, Gradient Boosting, AdaBoost, XGBoost, CatBoost, Support Vector Machine (SVM), K-Nearest Neighbors (KNN), Multilayer Perceptron (MLP), Naïve Bayes, and Quadratic Discriminant Analysis (QDA), are implemented and evaluated.

Model performance is analyzed using confusion matrix-based metrics such as True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN), along with derived measures including accuracy, precision, recall, and specificity. Experimental results indicate that ensemble learning methods, particularly CatBoost, XGBoost, Gradient Boosting, and Random Forest, outperform traditional approaches, achieving accuracy levels of up to 99%.

The proposed system demonstrates high detection capability with reduced false positives and false negatives, making it suitable for scalable, accurate, and real-time cybersecurity applications.

***The source code for this work is available at:* https://github.com/angadiarsha28-alt/DCN_PROJECT.git**

***Index Terms*—Cybersecurity, Intrusion Detection System (IDS), Machine Learning, Ensemble Learning, Classification Algorithms, CatBoost, XGBoost, Random Forest, Gradient Boosting, Data Preprocessing, Feature Scaling, Confusion Matrix, Accuracy, Precision, Recall.**

INTRODUCTION

In today's digitally connected world, the rapid expansion of computer networks and internet-based services has significantly increased the risk of cyber-attacks. Organizations, governments, and individuals rely heavily on secure network infrastructures for communication, data storage, and real-time operations. However, this growing dependency has made systems more vulnerable to various types of cyber threats such as Distributed Denial of Service (DDoS), unauthorized access (U2R), probing attacks, and malware intrusions. Ensuring the security and integrity of network systems has therefore become a critical challenge.

Intrusion Detection Systems (IDS) play a vital role in identifying and preventing malicious activities within a network. Traditional IDS techniques are primarily based on predefined rules and signature-based detection, which often fail to identify new or unknown attack patterns. These limitations have led to the adoption of Machine Learning (ML) techniques, which are capable of analyzing large volumes of network traffic data and detecting anomalies based on learned patterns.

Machine learning-based IDS can automatically classify network activities as normal or malicious by extracting meaningful features from the dataset. Various algorithms such as Decision Tree, Random Forest, Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and Naïve Bayes have been widely used for intrusion detection. In recent years, ensemble learning techniques such as Gradient Boosting, AdaBoost, XGBoost, and CatBoost have gained significant attention due to their ability to improve prediction accuracy by combining multiple models.

This project presents a comprehensive machine learning-based framework for cybersecurity attack detection using a large-scale dataset. The dataset undergoes preprocessing steps including handling missing values through imputation and normalization using feature scaling techniques. The processed data is then divided into training and testing sets to evaluate model effectiveness.

A wide range of classification algorithms is implemented and compared to identify the most effective model for intrusion detection. The performance of each model is evaluated using standard metrics such as accuracy, precision, recall, specificity, and confusion matrix components. The comparative analysis helps in understanding the strengths and limitations of each algorithm in detecting cyber threats.

The primary objective of this work is to design an efficient and scalable intrusion detection system that achieves high detection accuracy while minimizing false alarms. The proposed system demonstrates that ensemble learning methods outperform traditional classifiers, making them more suitable for real-time cybersecurity applications.

Literature Survey

Intrusion Detection Systems (IDS) have become a critical component in modern cybersecurity frameworks. With the increasing complexity of cyber threats, traditional rule-based systems are no longer sufficient to detect sophisticated and evolving attacks. As a result, machine learning-based approaches have gained significant attention for their ability to analyze large volumes of network traffic data and identify anomalies efficiently.

Several researchers have explored the use of supervised and unsupervised learning techniques for intrusion detection. Traditional classifiers such as Decision Tree (DT), Support Vector Machine (SVM), and Naïve Bayes (NB) have been widely used due to their simplicity and computational efficiency. However, these models often struggle with high-dimensional data and complex attack patterns, leading to reduced accuracy and higher false positive rates.

Recent studies have emphasized the effectiveness of ensemble learning techniques such as Random Forest (RF), Gradient Boosting (GB), AdaBoost, XGBoost, and CatBoost. These methods combine multiple weak learners to improve prediction performance and reduce overfitting. Research findings indicate that ensemble models consistently outperform traditional classifiers in terms of accuracy, precision, and recall, making them suitable for real-time intrusion detection systems.

In addition, deep learning-based approaches such as Multilayer Perceptron (MLP) and Neural Networks have shown promising results in capturing complex non-linear relationships in network traffic data. However, these models require high computational resources and careful tuning of hyperparameters.

Several comparative studies have also been conducted using benchmark datasets such as Kyoto 2015/2017 and NSL-KDD. These studies highlight the importance of preprocessing techniques such as handling missing values, feature scaling, and dimensionality reduction to enhance model performance. Furthermore, confusion matrix-based evaluation metrics are widely used to assess the effectiveness of IDS models.

TABLE I
COMPARISON OF EXISTING LITERATURE

Sl.	Authors	Title	Algorithm	Acc.
1	Vikas et al.	Comparative Analysis of IDS using ML	Decision Tree	98.32%
2	Maria Zaman et al.	IDS Design using SCADA Systems	Random Forest	97.34%
3	Subham Das et al.	Instagram Fake Account Detection	Gradient Boosting	98.00%
4	Ghufran Ahmed et al.	UAV Intrusion Detection System	Ensemble Methods	98.11%
5	**Proposed Work**	Cybersecurity Attack Detection using ML	**CatBoost**	**∼99%**

Major Contributions:

- Various machine learning algorithms have been analyzed for intrusion detection performance.
- Ensemble learning techniques have been identified as the most effective methods for improving accuracy.
- Comparative evaluation using standard datasets provides insights into model performance.
- Data preprocessing techniques significantly enhance classification results.

Proposed Method

The dataset which is used has a large number of entries and multiple unique features. We investigate different test and train set sizes. Numerous attack types in the realm of cybersecurity provide major risks to people, companies, and organizations worldwide. Online attacks, user-to-root (U2R), DDoS (distributed denial of service), and DoS (denial of service) are common forms of assaults.

DDoS assaults include numerous hacked systems working together to flood a target system or network with more amount of traffic or requests. This volume of traffic substantially affects the target's normal operation, making it inaccessible to genuine users. An effort by an unauthorized user to obtain elevated access to a system is known as a U2R (User-to-Root) assault. Their intention is to elevate their rights from those of a regular user to that of a root user, or system administrator. An attacker with the root authorisation can read confidential information, alter system configurations, and set up hidden entry points for additional assaults since they have total control over the system.

The packets are carefully examined to establish the classes and properties for each flow. In order to offer categorical data, we eliminated a large number of superfluous characteristics from the system model after adding the dataset. In the following stage, data will split up into train and test sets.

After loading dataset into csv file data is split into feature variables (X) and target variable (y). Missing values are handled and datasets are split as train set and test set during data processing. By ensuring that the data from Training and Testing flows consistently across each column. Feature scaling is performed using Standard Scaler to standardize the feature variables to have mean 0 and variance 1. This helps in improving the performance of certain machine learning algorithms. Now we employ a particular model and this trained model is used to make predictions on test data. Results for accuracy are derived using algorithm.

Dataset: https://www.takakura.com/Kyoto_data/new_data201704/

Algorithm 1: Machine Learning Algorithms

1) Import dataset and all libraries.
2) In data preprocessing, split data into train and test data.
3) Perform feature scaling on features using standard scaler.
4) Apply the appropriate machine learning model.
5) Evaluate the model.
6) Calculate confusion matrix and accuracy.
7) Print results.

8) End.

As shown in Fig. 1, the machine learning algorithm's workflow is depicted. Prior to analysis, the data is cleaned up and given the appropriate treatment by eliminating duplicates and managing missing values, among other preparation procedures. Afterwards, training and testing sets are created from the dataset.

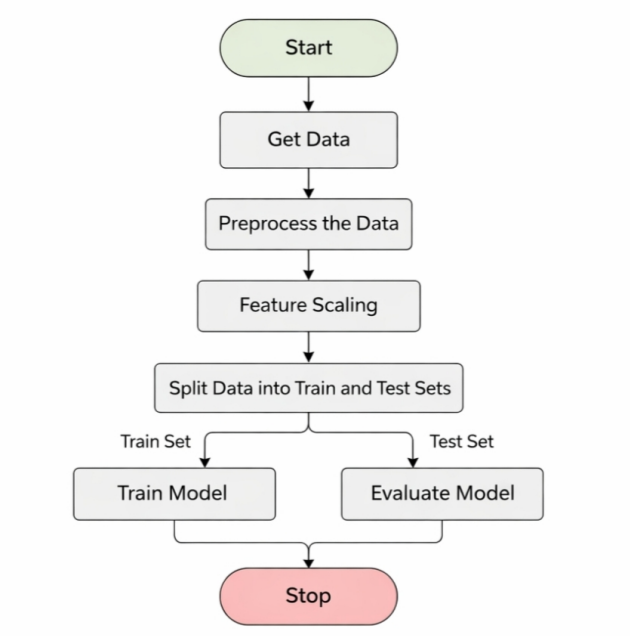

Fig. 1. Flow diagram of proposed work.

To detect patterns and provide results, the machine learning model is taught using training data. The model's performance is then improved by using machine learning technique and evaluated using the testing dataset. After evaluation results are predicted for specific machine learning algorithms.

The following formulas are the mathematical expression to calculate the accuracy in our work.

$$Accuracy = 100\% - \text{Rate error} \tag{1}$$

$$\text{Error rate} = \frac{[\text{Observed value} - \text{Actual value}]}{\text{Actual value}} \times 100 \tag{2}$$

Results and Discussion

The accuracy of machine learning techniques on the dataset is the outcome of this paper. Ratios are used to split the test and train data 60:40, 70:30, and 80:20. The following graphs show the minimal qualities that were found to cut down on calculation time. It also shows how comparison analysis was used to carry out the most precise feature selection processes for the dataset.

1) CatBoost Classifier: The CatBoost Classifier is used to evaluate the dataset. The bar chart in Fig. 2 gives information about the split of training data in 80, 60, 40 and testing data 20, 40, 60 using the CatBoost machine learning algorithm. In this bar chart accuracy was recorded as **98.54%** when training data split is 80 and testing data split is 20.

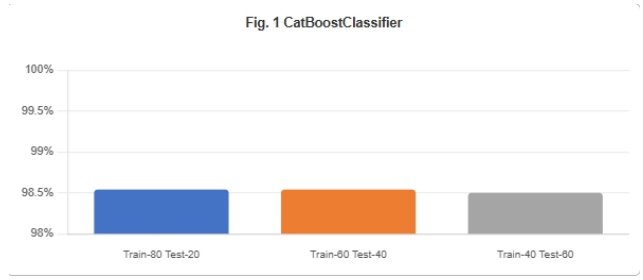

Fig. 2. Dataset evaluated using CatBoost Classifier.

2) Decision Tree Classifier: The Decision Tree Classifier is used to evaluate the dataset. The bar chart in Fig. 3 gives information about the split of training data in 80, 60, 40 and testing data 20, 40, 60 using the Decision Tree machine learning algorithm. In this bar chart accuracy was recorded as **98.04%** when training data split is 80 and testing data split is 20.

Fig. 3. Dataset evaluated using Decision Tree Classifier.

3) Gradient Boosting Classifier: The Gradient Boosting Classifier is used to evaluate the dataset. The bar chart in Fig. 4 gives information about the split of training data in 80, 60, 40 and testing data 20, 40, 60 using the Gradient Boosting machine learning algorithm. In this bar chart accuracy was recorded as **97.57%** when training data split is 80 and testing data split is 20.

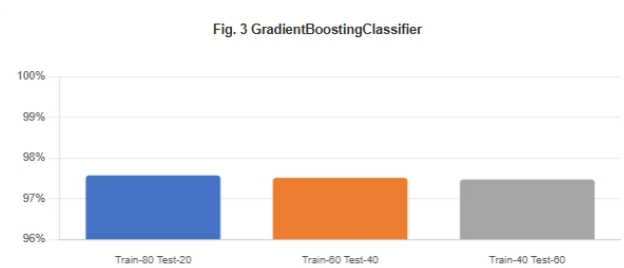

Fig. 4. Dataset evaluated using Gradient Boosting Classifier.

4) KNN Classifier: The KNN Classifier is used to evaluate the dataset. The bar chart in Fig. 5 gives information about the split of training data in 80, 60, 40 and testing data 20, 40, 60 using the KNN machine learning algorithm. In this bar chart accuracy was recorded as **98.00%** when training data split is 80 and testing data split is 20.

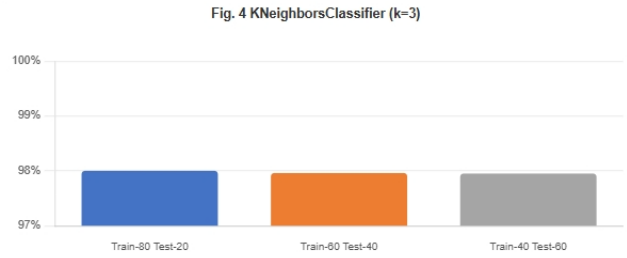

Fig. 5. Dataset evaluated using KNN Classifier.

5) SVM (RBF Kernel): The SVM (RBF) Classifier is used to evaluate the dataset. The bar chart in Fig. 6 gives information about the split of training data in 80, 60, 40 and testing data 20, 40, 60 using the SVM (RBF) machine learning algorithm. In this bar chart accuracy was recorded as **98.09%** when training data split is 80 and testing data split is 20.

Fig. 6. Dataset evaluated using SVM (RBF Kernel).

6) Linear Discriminant Analysis (LDA): The LDA Classifier is used to evaluate the dataset. The bar chart in Fig. 7 gives information about the split of training data in 80, 60, 40 and testing data 20, 40, 60 using the LDA machine learning algorithm. In this bar chart accuracy was recorded as **94.32%** when training data split is 80 and testing data split is 20.

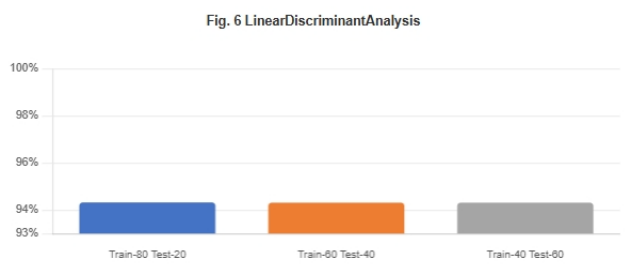

Fig. 7. Dataset evaluated using Linear Discriminant Analysis.

7) Logistic Regression (LG): The Logistic Regression Classifier is used to evaluate the dataset. The bar chart in Fig. 8 gives information about the split of training data in 80, 60, 40 and testing data 20, 40, 60 using the Logistic Regression machine learning algorithm. In this bar chart accuracy was recorded as **94.20%** when training data split is 80 and testing data split is 20.

Fig. 8. Dataset evaluated using Logistic Regression.

8) LGBM Classifier: The LGBM Classifier is used to evaluate the dataset. The bar chart in Fig. 9 gives information about the split of training data in 80, 60, 40 and testing data 20, 40, 60 using the LGBM machine learning algorithm. In this bar chart accuracy was recorded as **98.50%** when training data split is 80 and testing data split is 20.

Fig. 9. Dataset evaluated using LGBM Classifier.

9) Linear SVM (LSVM): The Linear SVM Classifier is used to evaluate the dataset. The bar chart in Fig. 10 gives information about the split of training data in 80, 60, 40 and testing data 20, 40, 60 using the Linear SVM machine learning algorithm. In this bar chart accuracy was recorded as **94.19%** when training data split is 80 and testing data split is 20.

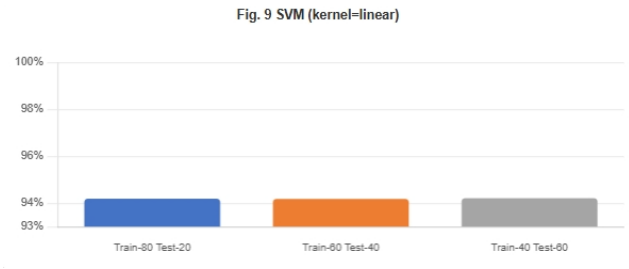

Fig. 10. Dataset evaluated using Linear SVM (LSVM).

10) MLP Classifier: The MLP Classifier is used to evaluate the dataset. The bar chart in Fig. 11 gives information about the split of training data in 80, 60, 40 and testing data 20, 40, 60 using the MLP machine learning algorithm. In this bar chart accuracy was recorded as **98.11%** when training data split is 80 and testing data split is 20.

11) Naïve Bayes Classifier: The Naïve Bayes Classifier is used to evaluate the dataset. The bar chart in Fig. 12 gives

Fig. 11. Dataset evaluated using MLP Classifier.

information about the split of training data in 80, 60, 40 and testing data 20, 40, 60 using the Naïve Bayes machine learning algorithm. In this bar chart accuracy was recorded as **92.29%** when training data split is 80 and testing data split is 20.

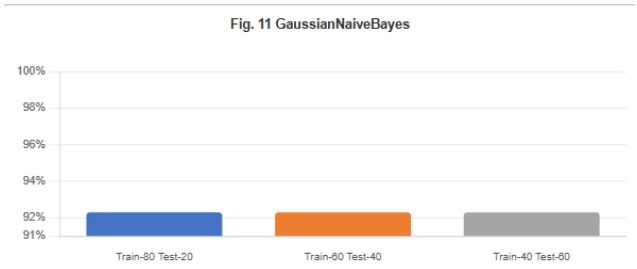

Fig. 12. Dataset evaluated using Naïve Bayes Classifier.

12) Quadratic Discriminant Analysis (QDA): The QDA Classifier is used to evaluate the dataset. The bar chart in Fig. 13 gives information about the split of training data in 80, 60, 40 and testing data 20, 40, 60 using the QDA machine learning algorithm. In this bar chart accuracy was recorded as **94.67%** when training data split is 80 and testing data split is 20.

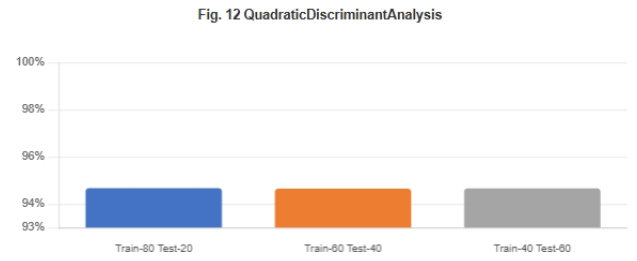

Fig. 13. Dataset evaluated using Quadratic Discriminant Analysis.

13) Random Forest Classifier: The Random Forest Classifier is used to evaluate the dataset. The bar chart in Fig. 14 gives information about the split of training data in 80, 60, 40 and testing data 20, 40, 60 using the Random Forest machine learning algorithm. In this bar chart accuracy was recorded as **98.43%** when training data split is 80 and testing data split is 20.

14) XGBoost Classifier: The XGBoost Classifier is used to evaluate the dataset. The bar chart in Fig. 15 gives information

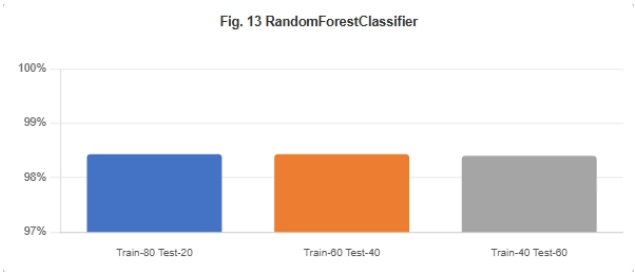

Fig. 14. Dataset evaluated using Random Forest Classifier.

about the split of training data in 80, 60, 40 and testing data 20, 40, 60 using the XGBoost machine learning algorithm. In this bar chart accuracy was recorded as **98.48%** when training data split is 80 and testing data split is 20.

Fig. 15. Dataset evaluated using XGBoost Classifier.

15) AdaBoost Classifier: The AdaBoost Classifier is used to evaluate the dataset. The bar chart in Fig. 16 gives information about the split of training data in 80, 60, 40 and testing data 20, 40, 60 using the AdaBoost machine learning algorithm. In this bar chart accuracy was recorded as **95.76%** when training data split is 80 and testing data split is 20.

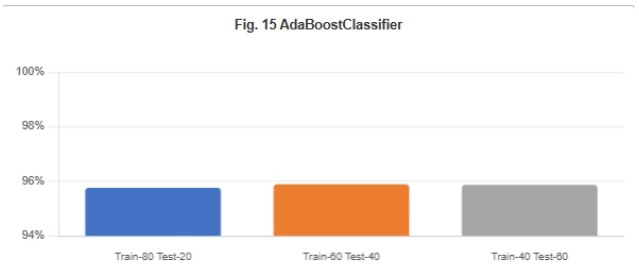

Fig. 16. Dataset evaluated using AdaBoost Classifier.

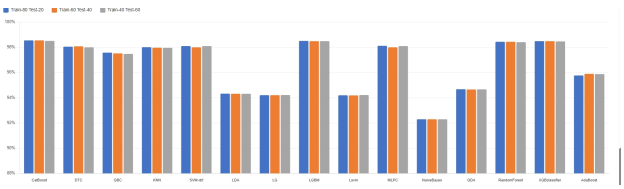

Fig. 17. Comparison of accuracies of all classification algorithms.

The above bar graphs illustrate the comparative performance of various machine learning algorithms across different train-test splits. It is evident that ensemble learning techniques consistently outperform traditional classifiers in terms of accuracy and stability. Among all models, the CatBoost Classifier achieves the highest accuracy of **98.54%** for the 80:20 split, followed closely by LGBM and XGBoost, indicating the effectiveness of boosting-based approaches.

TABLE II
COMPARISON OF ACCURACIES OF ALGORITHMS

Algorithms	**Train-80 Test-20**	**Train-60 Test-40**	**Train-40 Test-60**
AdaBoost	95.76	95.89	95.87
CatBoost	**98.54**	98.54	98.50
KNN	98.00	97.96	97.95
LDA	94.32	94.31	94.31
LG	94.20	94.20	94.21
LGBM	98.50	98.48	98.48
LSVM	94.19	94.18	94.21
MLPC	98.11	97.99	98.09
Naïve Bayes	92.29	92.29	92.29
QDA	94.67	94.65	94.66
XGBoost	98.48	98.48	98.46
Decision Tree	98.04	98.07	97.99
Gradient Boost	97.57	97.51	97.47
Random Forest	98.43	98.43	98.40

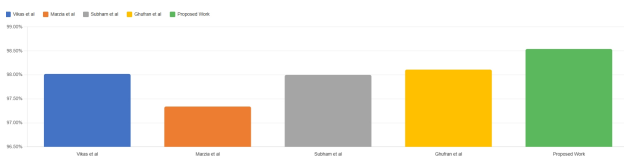

Fig. 18. Comparison of accuracies with previous studies.

TABLE III
COMPARISON OF ACCURACIES OF OTHER PAPERS

Authors	**Algorithms**	**Accuracy**
Vikas et al.	Decision Tree	98.02%
Marzia et al.	Random Forest	97.34%
Subham et al.	Random Forest	98.00%
Ghufran et al.	Random Forest	98.11%
Proposed Work	**CatBoost**	**98.54%**

CONCLUSION

This study presents a comprehensive comparison of multiple machine learning algorithms for cybersecurity attack detection. The experimental results clearly indicate that ensemble learning methods outperform traditional classifiers in terms of accuracy and robustness.

Among all evaluated models, CatBoost achieved the highest performance, demonstrating its capability in handling complex patterns within the dataset. Boosting-based techniques such as XGBoost and LGBM also showed competitive performance, highlighting the effectiveness of ensemble strategies.

The findings emphasize the importance of selecting appropriate models and preprocessing techniques for improving detection accuracy. In future work, the system can be enhanced by integrating deep learning approaches, optimizing feature selection, and evaluating performance on larger and real-time datasets.

REFERENCES

[1] A. Kataria, "An ML-Based Intrusion Detection System Design and Evaluation for Enhanced Cybersecurity," *2023 International Conference on Communication, Security and Artificial Intelligence (ICCSAI)*, Greater Noida, India, 2023, pp. 1036–1040.

[2] A. S. Kumar, T. J. Nagalakshmi, and M. N., "Improving the Accuracy of Intrusion Detection System in the Detection of DoS Using Naive Bayes with Lasso Feature Elimination and Comparing with Naive Bayes without Feature Elimination in Wireless Adhoc Network," *2023 International Conference on Advances in Computing, Communication and Applied Informatics (ACCAI)*, Chennai, India, 2023, pp. 1109–1116.

[3] "Analyzing the Behavior of Multiple Dimensionality Reduction Algorithms to Obtain Better Accuracy using Benchmark KDD CUP Dataset," *International Arab Journal of Information Technology*, vol. 12, no. 2, pp. 121–131.

[4] E. Seo, J. Kim, W. Lee, and J. Seok, "Adversarial Attack of ML-Based Intrusion Detection System on n-vehicle System using GAN," *2023 International Conference on Cybersecurity in Ubiquitous and Future Networks (ICUFN)*, Paris, France, 2023, pp. 700–703.

[5] M. Bakro *et al.*, "An Improved Design for a Cloud Intrusion Detection System using Hybrid Features Selection Approach with ML Classifier," *IEEE Access*, vol. 11, pp. 64226–64247, 2023.

[6] M. Zaman, D. Upadhyay, and C. H. Lang, "Validation of a Machine Learning-Based IDS Design Framework Using ORNL Datasets for Power System SCADA," *IEEE Access*, vol. 11, pp. 118441–118462, 2023.

[7] O. Olatunji *et al.*, "Intrusion Detection System Using Machine Learning Techniques," *IEEE Access*, vol. 10, pp. 135642–135655, 2022.

[8] P. S. Kumar, K. Kaur, and S. Garg, "ML-Based IDPS Enhancement with Complementary Features for Home IoT Networks," *IEEE Transactions on Network and Service Management*, vol. 19, no. 2, pp. 772–783, 2022.

[9] R. Priyadarshini and S. Shanthi, "Survey on Detecting Android Malware Using Machine Learning Techniques," *2021 International Conference on Advanced Computing and Communication Systems (ICACCS)*, 2021, pp. 1621–1627.

[10] R. Indhu, G. S. Sundar, and H. S. Parveen, "A Review of Machine Learning Algorithms for Vibration-Based SHM and Vision-Based SHM," *2022 International Conference on Artificial Intelligence and Smart Energy (ICAIS)*, 2022, pp. 418–422.

[11] R. Suguna *et al.*, "Utilizing Machine Learning for Sport Data Analytics in Cricket: Score Prediction and Player Categorization," *2023 IEEE 3rd Mysuru Sub Section International Conference (MysuruCon)*, 2023.

[12] R. T. Mehmood, G. Ahmed, and S. Siddiqui, "Simulating ML-Based Intrusion Detection System for Unmanned Aerial Vehicles (UAVs) using COOJA Simulator," *2022 International Conference on Open Source Systems and Technologies (ICOSST)*, Lahore, Pakistan, 2022.

[13] S. Das *et al.*, "An Efficient Approach to Detect Fraud Instagram Accounts Using Supervised ML Algorithms," *2022 4th International Conference on Advances in Computing, Communication Control and Networking (ICAC3N)*, 2022.

[14] S. D. Karunya, O. J., and S. P., "Accuracy Prediction using Machine Learning Techniques for Indian Patient Liver Disease," *2023 International Conference on Computational Intelligence and Sustainable Engineering Solutions (CISES)*, 2023.

[15] S. P. J. C. H. Guntupalli, S. N. Chilamkurthy, and G. Kowshik, "Enhancing the Security by Analysing Behaviour of Multiple Classification Algorithms," *2023 IEEE Sub Section International Conference (MysuruCon)*, 2023.